OOP Fundamentals

What is Object-Oriented Programming

OOP is a programming paradigm built around 'objects' — bundles of data (attributes) and behavior (methods) — rather than just functions and logic acting on raw data. Languages like Java, C++, and Python support OOP. It's used to model real-world entities in a structured, reusable way.

The Four Pillars of OOP

- Encapsulation: bundling data and the methods that operate on it into a single unit (a class), while restricting direct access to internal details using access modifiers (private, protected, public).
- Abstraction: hiding complex implementation details and exposing only what's necessary through a simple interface — you use a car's steering wheel without knowing the engine's internals.
- Inheritance: a class (child/subclass) can acquire properties and methods from another class (parent/superclass), enabling code reuse and hierarchical relationships.
- Polymorphism: the same method name behaves differently depending on the object calling it — either through method overriding (child class redefines a parent method) or method overloading (same method name, different parameters).

Class vs Object

A class is a blueprint/template that defines attributes and methods. An object is a specific instance created from that class, with its own actual data. You can create many objects from one class — e.g. a 'Car' class might produce 'myHonda' and 'yourToyota' as two separate objects, each with their own color, speed, etc.

Encapsulation in Python

```
class BankAccount:
    def __init__(self, balance):
        self.__balance = balance   # double underscore = private-ish in Python

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount(1000)
```

```
account.deposit(500)
print(account.get_balance())   # 1500
# account.__balance would not work directly from outside the class
```

Inheritance in Python

```
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        return "Some generic sound"

class Dog(Animal):          # Dog inherits from Animal
    def speak(self):        # method overriding
        return f"{self.name} says Woof!"

class Cat(Animal):
    def speak(self):
        return f"{self.name} says Meow!"

animals = [Dog("Rex"), Cat("Whiskers")]
for a in animals:
    print(a.speak())   # polymorphism: same method call, different behavior
```

Abstraction in Python

```
from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self):
        pass   # no implementation - forces subclasses to define it
```

```
class Rectangle(Shape):
    def __init__(self, w, h):
        self.w, self.h = w, h
    def area(self):
        return self.w * self.h

# Shape() directly would raise an error - it's abstract
r = Rectangle(4, 5)
print(r.area())  # 20
```

Method Overloading vs Overriding

- Overriding: a subclass provides its own implementation of a method already defined in its parent class (same name, same signature) — runtime polymorphism.
- Overloading: multiple methods with the same name but different parameters within the same class — Python doesn't support true overloading natively (the last definition wins), unlike Java/C++, but default arguments can simulate it.

Common Interview Questions

- Explain the four pillars of OOP with real examples.
- What is the difference between an abstract class and an interface?
- Can a constructor be private? Why would you do that (Singleton pattern)?
- What is method overriding vs overloading, and does Python support both?
- What is the difference between composition and inheritance ('has-a' vs 'is-a')?
- What is multiple inheritance, and what problems can it cause (Diamond Problem)?